Project AETHER: Autonomous Constellation Management System

National Space Hackathon 2026

Orbital Debris Avoidance & Constellation Management

Team Syntaxion

March 31, 2026

Abstract

As Low Earth Orbit (LEO) becomes increasingly congested with commercial mega-constellations and millions of debris fragments, traditional human-in-the-loop collision avoidance systems are no longer scalable. This report details the architecture and implementation of Project AETHER, an Autonomous Constellation Manager (ACM) designed to handle high-frequency telemetry, perform predictive conjunction assessment using KD-Trees, and execute optimal collision avoidance and recovery maneuvers using Proximal Policy Optimization (PPO). Our system adheres to J2-perturbed orbital mechanics and strict propellant constraints, ensuring both safety and mission uptime.

Contents

1 Introduction **3**

2 System Architecture **3**

2.1 Backend Components 3

2.2 Frontend: Orbital Insight Visualizer 3

3 Numerical Methods and Orbital Mechanics **3**

3.1 Equations of Motion with J2 Perturbation 3

3.2 Numerical Integration (RK4) 4

3.3 Propulsion and Fuel Mass Depletion 4

4 Spatial Optimization and Conjunction Assessment **4**

4.1 CDMs and TCA Calculation 4

5 Autonomous Maneuvering and AI Logic **4**

5.1 Proximal Policy Optimization (PPO) 4

5.2 Physics-Informed Neural Networks (PINNs) 5

5.3 End-of-Life (EOL) & Graveyard Orbit Management 5

5.4 Strategic Lookahead and Performance 5

5.5 Station-Keeping and Recovery 5

6 Conclusion **5**

1 Introduction

The rapid expansion of space-based services has led to a critical density of objects in LEO, bringing the threat of Kessler Syndrome to the forefront of orbital operations. Project AETHER (Autonomous Evasion and Trajectory Handling for Extra-atmospheric Risk) aims to modernize satellite operations by shifting from ground-reliant piloting to onboard autonomy.

The primary challenges addressed include:

- **Scalability**: Processing thousands of satellites and debris fragments.
- **Latency**: Operating through communication blackout zones.
- **Efficiency**: Minimizing Δv consumption while maintaining orbital slots.

2 System Architecture

The Project AETHER software suite is built as a distributed microservice architecture, containerized using Docker for reproducible deployment.

2.1 Backend Components

- **Telemetry Processor**: Handles asynchronous ingestion of ECI state vectors at port 8000.
- **Physics Engine**: Propagates objects using 4th Order Runge-Kutta (RK4) integration with J2 perturbation modeling.
- **Autonomous Controller**: A deep reinforcement learning agent (PPO) and a Physics-Informed Neural Network (PINN) that coordinates maneuvers and predicts trajectories.
- **Spatial Indexing Layer**: Uses SciPy-based KD-Trees for $O(\log N)$ proximity searches.

2.2 Frontend: Orbital Insight Visualizer

A real-time 2D Ground Track visualizer using Canvas/WebGL to display the sub-satellite points, historical trails, and predicted trajectories.

3 Numerical Methods and Orbital Mechanics

3.1 Equations of Motion with J2 Perturbation

Orbits are modeled using the Earth-Centered Inertial (ECI) coordinate frame. The acceleration vector $\vec{a}$ includes the primary gravitational term and the J2 perturbation due to Earth's oblateness:

$$\frac{d^2\vec{r}}{dt^2} = -\frac{\mu}{|\vec{r}|^3}\vec{r} + \vec{a}_{J2} \tag{1}$$

The J2 acceleration $\vec{a}_{J2}$ accounts for nodal regression and apsidal precession, critical for long-term station-keeping:

$$\vec{a}_{J2} = \frac{3}{2}\frac{J_2 \mu R_E^2}{|\vec{r}|^5}\left[\begin{pmatrix} x(\frac{5z^2}{|\vec{r}|^2} - 1) \\ y(\frac{5z^2}{|\vec{r}|^2} - 1) \\ z(\frac{5z^2}{|\vec{r}|^2} - 3) \end{pmatrix}\right] \tag{2}$$

3.2 Numerical Integration (RK4)

We employ a 4th Order Runge-Kutta method for high-fidelity propagation of state vectors:

$$
\begin{aligned}
k_1 &= f(t_n, y_n) \\
k_2 &= f(t_n + \frac{h}{2}, y_n + h\frac{k_1}{2}) \\
k_3 &= f(t_n + \frac{h}{2}, y_n + h\frac{k_2}{2}) \\
k_4 &= f(t_n + h, y_n + hk_3) \\
y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)
\end{aligned}
$$

3.3 Propulsion and Fuel Mass Depletion

Thruster burns are modeled as impulsive changes in velocity ($\Delta\vec{v}$). Propellant consumption follows the Tsiolkovsky rocket equation:

$$\Delta m = m_{current}\left(1 - e^{-\frac{|\Delta\vec{v}|}{I_{sp}g_0}}\right) \tag{3}$$

We strictly enforce a 600-second thermal cooldown between consecutive burns on the same satellite.

4 Spatial Optimization and Conjunction Assessment

To avoid $O(N^2)$ bottlenecks during conjunction assessment, AETHER implements a KD-Tree spatial index.

4.1 CDMs and TCA Calculation

The system generates Conjunction Data Messages (CDMs) whenever a debris fragment is predicted to enter a 100-meter danger sphere around a satellite.

- **Indexing**: All debris positions are stored in a KD-Tree.
- **Querying**: Satellites query the tree for nearest neighbors within a defined threat radius (e.g., 5 km).
- **TCA Determination**: For identified threats, time of closest approach (TCA) is calculated via iterative minimization of the relative distance vector.

5 Autonomous Maneuvering and AI Logic

5.1 Proximal Policy Optimization (PPO)

Evasion maneuvers are determined by a PPO reinforcement learning agent. The state space includes the satellite's relative position/velocity to the threat, remaining fuel, and slot deviation.

- **Actor-Critic Architecture**: Two 256-unit hidden layer networks process inputs to output optimal Δv vectors in the Radial-Transverse-Normal (RTN) frame.

- **Reward Function**: Penalizes collisions severely and penalizes Δv consumption and slot drift.

5.2 Physics-Informed Neural Networks (PINNs)

To enhance trajectory prediction, we implement a PINN model (`OrbitPINN`). Unlike traditional black-box neural networks, our PINN incorporates the equations of motion directly into its loss function:

$$\mathcal{L}_{physics} = \left\| \frac{d\vec{r}}{dt} - \vec{v}_{pred} \right\|^2 + \left\| \frac{d\vec{v}}{dt} - \vec{a}_{total_physics} \right\|^2 \tag{4}$$

This ensures that the predicted state vectors $S(t) = [x, y, z, v_x, v_y, v_z]$ strictly obey the J2-perturbed gravitational laws, even when data is sparse or noisy.

5.3 End-of-Life (EOL) & Graveyard Orbit Management

Adhering to the 5% fuel depletion constraint, AETHER autonomously detects EOL status. Upon reaching this threshold, the system schedules a one-time prograde burn (e.g., +5 m/s) to raise the satellite into a stable graveyard orbit, mitigating future collision risks.

5.4 Strategic Lookahead and Performance

To maintain high-frequency telemetry processing without $O(N^2)$ lag, our `AutoPilot` utilizes:

- **Strategic Lookahead**: Preemptively calculates conjunction risks up to 24 hours in advance.
- **Asynchronous Threading**: Numerical integration and threat analysis are delegated to thread pools via `asyncio.to_thread`, preventing event-loop blocking.

5.5 Station-Keeping and Recovery

After an evasion maneuver, the system calculates a recovery burn to return the satellite to its nominal orbital slot (±10 km radius). This ensures high constellation uptime even under constant debris threats.

6 Conclusion

Project AETHER provides a robust, scalable framework for autonomous space traffic management. By integrating high-fidelity orbital mechanics with cutting-edge AI and spatial optimization, we demonstrate a path toward sustainable and safe LEO operations for the next generation of mega-constellations.